ChatGPT

책으로만 읽은 팔란티어와 온톨로지 이야기

부제: 검사·계측 AI 분석은 어떻게 조직의 판단 체계가 될 수 있는가

Executive Summary

이 보고서의 핵심 메시지는 다섯 가지다.

- 팔란티어를 단순한 방산 회사, 데이터 분석 회사, 또는 AI 모델 회사로만 보면 핵심을 놓치기 쉽다. 공식 문서 기준으로 팔란티어의 제품 묶음은 Gotham, Foundry, Apollo, AIP이며, 이들은 데이터 통합, 논리 작성, 온톨로지 개발, 운영 워크플로, 그리고 생성형 AI를 하나의 운영 아키텍처로 묶는다. 팔란티어는 이를 사실상 "Enterprise Operating System"에 가깝게 설명한다. [1]
- 정보학에서의 온톨로지는 단순 스키마보다 넓다. 그것은 어떤 대상이 있고, 어떤 속성을 가지며, 무엇과 어떤 관계를 맺고, 어떤 제약과 규칙 아래 해석되어야 하는지를 명시하는 지식표현 체계다. W3C의 OWL은 이를 형식 언어로 제공하고, SKOS는 분류체계·택소노미를 더 가벼운 수준에서 표현한다. [2]
- 팔란티어의 Ontology는 이 전통적 의미의 온톨로지를 기업 운영에 맞게 확장한 형태다. Palantir 공식 문서는 Ontology를 조직의 디지털 자산 위에 놓인 "operational layer"이자 "digital twin"으로 설명하며, 객체·속성·링크·액션·함수·권한을 함께 묶어 실제 업무 변경과 의사결정을 가능하게 한다. [3]
- 마르쿠스 가브리엘의 의미장 이론은 회사 온톨로지를 이해하는 데 유용한 해석 프레임을 준다. 같은 Wafer라도 생산 의미장에서는 공정 진행 단위이고, 검사·계측 의미장에서는 결함과 측정값의 귀속 대상이며, 수율 의미장에서는 집계 단위가 된다. 즉, 좋은 회사 온톨로지는 테이블 정리보다 "조직이 실제로 무엇을 같은 대상으로 보고 있는가"를 정리하는 일에 가깝다. [4]
- 반도체 검사·계측 AI 분석 조직에서 더 중요한 질문은 "모델 정확도가 몇 % 더 올랐는가"만이 아니다. 더 근본적인 질문은 "이상 탐지 결과가 case open, 담당자 피드백 요청, hold/review 제안, 재검사, case close, 재학습으로 닫히는가"이다. Ontology는 바로 그 operational loop를 설계하는 데 유리하다. [5]

전체 논리 흐름 요약
먼저 팔란티어가 시장에서 왜 주목받는지 살핀다. 그다음 "존재론–정보학 온톨로지–팔란티어 Ontology"를 구분해 개념의 혼선을 푼다. 이어서 마르쿠스 가브리엘의 의미장 이론을 빌려, 회사 온톨로지가 왜 "DB 표준화"보다 "업무 세계 정의"에 가까운지 설명한다. 그 위에서 Palantir Ontology의 구조를 공식 문서로 정리하고, ERP/MES/BI/MLOps/Knowledge Graph와의 차이를 비교한다. 마지막으로 반도체 검사·계측 AI 조직의 업무 흐름에 이를 이식하면 어떤 객체, 관계, 함수, 액션을 최소 단위로 설계할 수 있는지 제안한다. [6]

이 보고서의 최종 주장
이 보고서의 최종 주장은 다음과 같다. **반도체 검사·계측 AI의 경쟁력은 모델 그 자체보다, 모델 결과를 조직의 판단·권한·행동·피드백 루프로 묶어 축적 가능한 운영 체계로 만드는 데서 나온다.** 팔란티어의 Ontology는 그 점을 가장 선명하게 보여주는 사례이며, 우리에게 필요한 것은 "팔란티어를 도입하자"가 아니라 "우리 조직의 검사·계측 의미장을 객체·관계·판단·액션으로 명시하자"는 문제의식이다. [7]

팔란티어는 왜 주목받는가

핵심 메시지
팔란티어를 이해할 때 가장 중요한 질문은 "무슨 업종인가?"가 아니라 "무엇을 운영 가능하게 만드는가?"다. 공식 자료를 보면 팔란티어는 데이터 수집 업체라기보다, 데이터·업무 로직·보안·배포·AI를 연결해 실시간 운영 결정을 가능하게 하는 플랫폼 기업으로 읽는 편이 더 정확하다. [8]

상세 설명
팔란티어는 2003년에 설립되었고, 초기에는 미국 정보기관의 대테러 수사와 작전을 지원하는 소프트웨어를 만들었다. 이후 상업 고객으로 확장했는데, 회사의 2025년 10-K는 정부 고객과 상업 고객이 "데이터를 다루는 근본적으로 유사한 문제"를 가진다고 설명한다. 2025년 연간 매출은 44억7544만 달러였고, 이 중 정부 매출은 24억228만 달러, 상업 매출은 20억7316만 달러였다. 2025년 미국 매출 비중은 전체의 74%였다. [9]

제품 구조도 단순하지 않다. 10-K와 공식 아키텍처 문서에 따르면, Gotham은 국방·정보 임무를 포함한 고위험 운영 영역에서 쓰이는 플랫폼이고, Foundry는 데이터 관리·논리 작성·온톨로지 개발·분석·워크플로를 담당하는 기반층이다. Apollo는 배포와 운영을 담당하는 연속 전달 플랫폼이며, AIP는 LLM 연결, 에이전트·자동화 개발, Evals, 운영 환경 내 거버넌스를 담당하는 생성형 AI 계층이다. 팔란티어는 이 셋을 합쳐 "Enterprise Operating System"으로 설명한다. [10]

시장이 팔란티어에 집중한 이유는 제품 철학 하나 때문만은 아니다. 숫자도 있었다. 2026년 1분기 Business Update 기준 팔란티어의 분기 매출은 16억3000만 달러로 전년 동기 대비 85% 증가했고, 상업 매출은 7억7400만 달러, 정부 매출은 8억5800만 달러였다. 특히 미국 상업 매출은 5억9500만 달러로 전년 동기 대비 133% 증가했고, 전체 고객 수는 1007개, 미국 상업 고객 수는 615개였다. 같은 자료에서 팔란티어는 1분기에 100만 달러 이상 계약 206건, 500만 달러 이상 계약 72건, 1000만 달러 이상 계약 47건을 체결했다고 밝혔다. [11]

정부·국방 영역의 존재감도 여전하다. 미 국방부 계약 공지에 따르면 2025년 5월 미 육군은 Maven Smart System 소프트웨어 라이선스에 대해 7억9500만 달러 규모의 수정 계약을 발표했다. 이어 2025년 7월에는 기존 여러 계약을 통합하는 기업형 계약이 발표되었고, 공식 공지상 계약 액수는 100억 달러였다. 다만 Reuters 보도는 이 계약이 향후 10년간 "최대" 100억 달러까지 구매할 수 있게 하는 구조이며, 그 전체 금액 집행이 자동으로 보장되는 것은 아니라고 설명했다. 이 구분은 매우 중요하다. **계약 ceiling과 인식 매출은 다르다.** [12]

주식시장의 관심도 이러한 맥락을 반영했다. S&P Dow Jones Indices는 2024년 9월 Palantir의 S&P 500 편입을 발표했고, 2025년 3월에는 S&P 100 편입도 공지했다. 2026-06-24 기준 시장 데이터에서 PLTR의 주가는 116.7달러, 시가총액은 약 3000억 달러로 집계되었다. 이는 투자 추천의 근거가 아니라, 기업이 시장에서 "AI + 운영 플랫폼 + 국방/정부 수요"의 교차점으로 인식되고 있음을 보여주는 배경 자료로 이해해야 한다. [13]

팔란티어의 핵심을 "AI 회사"라고만 부르기 어려운 이유도 여기 있다. LLM 기능 자체는 다른 기업도 제공할 수 있다. 그러나 팔란티어는 공식 문서에서 AIP를 Foundry와 Apollo 위에 올리고, 그 사이를 Ontology가 연결하는 구조로 설명한다. 다시 말해, 팔란티어가 파는 것은 모델 하나가 아니라 **조직의 데이터·권한·업무 로직·의사결정·액션까지 묶는 운영 문법**에 가깝다. 이 해석은 공식 문서의 구조에서 나오는 합리적 추론이다. [14]

플랫폼 자체보다 시장의 조명이 먼저 들어왔다는 점을 보는 데는 주가 차트가 유용하다. 다만 이것은 제품 우열이 아니라 시장 관심의 배경 자료다. 2026-06-24 기준 가격과 시가총액은 아래와 같다. [15]

객관적 근거와 출처
이 섹션의 핵심 근거는 팔란티어 2025년 10-K, 2026년 1분기 Business Update, Palantir 공식 아키텍처 문서, S&P 공지, 미국 국방부 계약 공지, 그리고 현재 시장 데이터다. 특히 제품 정의와 제품 간 관계는 Palantir 공식 문서에, 재무와 세그먼트는 SEC 및 IR 자료에, 국방 계약은 DoD 공지에 근거했다. [16]

반도체 검사·계측 AI 분석 업무와의 연결점
반도체 현장 관점에서 팔란티어를 흥미롭게 만드는 것은 "AI를 잘한다"는 추상적 문장이 아니다. 더 중요한 것은 제조 현장의 데이터, 분석 로직, 권한, 승인, 조치 이력을 하나의 운영 컨텍스트로 엮는 방식이다. 검사·계측 AI 조직도 비슷한 문제를 가진다. 이미지 모델, 결함 지도 분석, 계측값 이상 탐지, 리뷰 좌표 추천이 각각 따로 존재하면 운영 효율은 제한된다. 그러나 그것들이 Wafer, Lot, Equipment, Process Step, Case, Feedback Request, Action Item으로 연결되면 비로소 하나의 판단 체계가 된다. 이 점에서 팔란티어는 제품 그 자체보다 문제 정의 방식이 더 중요하다. [17]

청중이 헷갈릴 수 있는 지점과 방어 논리
가장 흔한 오해는 “팔란티어는 결국 데이터 통합 SI 아니냐”는 것이다. 이 말은 절반만 맞다. 데이터 통합은 분명 중요한 기능이지만, 공식 문서에서 온톨로지·액션·함수·권한·AI 실행까지 함께 설명되는 만큼, 이를 단순 ETL/SI 범주로 축소하면 부족하다. 반대로 “팔란티어는 곧 AI 모델 회사”라는 말도 과장이다. Palantir 문서에서 AIP는 LLM 연결과 에이전트 개발을 제공하지만, 그것은 Foundry와 Ontology 위에서 작동하도록 설계되어 있다. 따라서 더 적절한 표현은 **운영형 AI 플랫폼**이다. [18]

시각화 아이디어
“팔란티어는 무엇을 하는 회사인가?”라는 질문형 도식이 좋다. 왼쪽에는 방산/데이터 분석/AI/DevOps/SI라는 흔한 분류를 두고, 오른쪽에는 Gotham–Foundry–Apollo–AIP–Ontology 구조를 배치한 뒤 가운데 화살표를 “operational AI platform”으로 연결하면 청중의 오해를 정리하기 쉽다. 보조 그림으로는 2025년 세그먼트 매출 비중과 2026년 1분기 성장률 막대그래프가 적합하다. [19]

존재론, 온톨로지, 팔톨로지

핵심 메시지
이 보고서에서 가장 먼저 정리해야 할 혼선은 같은 영어 단어 ontology가 서로 다른 층위에서 쓰인다는 점이다. 철학에서는 존재 일반의 구조를 다루고, 정보학에서는 도메인 개념과 관계를 형식화하며, 팔란티어에서는 그것을 운영과 액션까지 확장한다. 이 세 층위를 구분하지 않으면 이후 논의가 계속 어긋난다. [20]

다음 표는 이 보고서의 용어 구분이다. 여기서 **“팔톨로지”**는 Palantir Ontology를 설명하기 위해 이 보고서에서 편의상 쓰는 표현이며, Palantir의 공식 제품명이 아니다.

구분	명칭	핵심 질문	설명
철학	존재론	무엇이 존재하는가	존재자, 범주, 관계, 실재의 가장 일반적 구조를 묻는 분야
정보학	온톨로지	업무 세계를 어떤 개념과 관계로 표현할 것인가	개념, 속성, 관계, 제약을 명시적으로 정의하는 지식 표현 체계
팔란티어	팔톨로지	조직의 판단과 행동을 어떤 객체·관계·행위로 실행할 것인가	데이터 모델을 넘어 액션, 함수, 권한, 거버넌스, AI 실행 문맥까지 포함하는 운영 계층

이 구분은 철학의 존재론을 과소평가하지 않으면서도, 기업 소프트웨어의 Ontology를 과도하게 형이상학화하지 않게 도와준다. SEP에 따르면 철학적 ontology는 현실의 근본 구성요소와 범주를 식별하려는 시도이고, 정보시스템에서의 ontology는 범주 연구를 공유하지만 “형식 구문과 형식 의미론을 통해 범주의 목록을 기록”하는 실용적 목적을 가진다. [21]

정보학에서의 온톨로지를 가장 널리 인용되는 방식으로 설명하면, Gruber는 ontology를 개념화의 명시적 명세로, 후속 전통은 이를 “공유된” 그리고 “형식적” 명세로 확장해 이해해 왔다. W3C OWL은 클래스, 속성, 개체, 데이터 값을 포함한 형식 언어를 제공하고, RDF와 함께 사용될 수 있다. 즉 온톨로지는 그저 테이블명 사전이 아니라, 대상과 관계의 의미를 기계가 해석 가능한 수준으로 올리는 표현 체계다. [22]

여기서 taxonomy, schema, data model, ontology를 구분할 필요가 있다. W3C SKOS는 taxonomy를 포함한 분류 체계를 비교적 가볍게 표현하는 RDF 어휘다. taxonomy는 보통 상하위 분류 구조에 강하지만, 풍부한 제약·규칙·추론까지 모두 담지는 않는다. 반면 ontology는 개념 간 의미 관계와 제약을 더 풍부하게 담는다. 데이터 모델이나 스키마는 저장과 조인, 타입, 무결성 같은 구조를 정의하는 데 강하다. Palantir 공식 문서가 object type을 dataset, object를 row, property를 column, link type을 join에 비유하는 것도 이 차이를 잘 보여준다. 즉 **스키마는 저장의 관점**, **온톨로지는 의미의 관점**, **팔란티어 Ontology는 의미+행동의 관점**으로 이해하면 된다. [23]

Knowledge Graph와의 관계도 분리해서 보는 편이 좋다. IBM은 knowledge graph를 현실의 엔티티와 관계를 그래프로 표현한 것으로 설명하고, ontologies가 그 graph의 엔티티를 형식적으로 표현하는 역할을 한다고 정리한다. 실무적으로는 ontology가 “무엇이 어떤 종류인가”를 정의하고, knowledge graph가 “실제 인스턴스들이 어떻게 연결되는가”를 담는 경우가 많다. 둘은 자주 함께 쓰이지만 같은 개념은 아니다. 24

팔란티어 Ontology는 이보다 한 단계 더 나간다. Palantir 공식 문서는 Ontology를 조직의 세계를 객체·속성·링크·액션 타입으로 매핑한 디지털 트윈으로 설명하며, 역할과 권한, 함수, 사용자 액션까지 포함한다고 말한다. 그래서 이 보고서에서는 같은 영어 단어를 계속 반복하기보다, 철학은 “존재론”, 정보학은 “온톨로지”, Palantir는 “팔톨로지”로 구분해 쓰는 편이 유용하다. 이 구분의 목적은 학술적 엄밀성을 흉내 내기 위함이 아니라, 청중이 서로 다른 층위의 논의를 섞어버리는 것을 막기 위함이다. 25

객관적 근거와 출처
철학적 ontology의 정의는 Stanford Encyclopedia of Philosophy에, 정보학 ontology의 전통적 정의는 Gruber와 Guarino 계열 문헌에, RDF/OWL 및 taxonomy 표현의 차이는 W3C OWL과 SKOS 문서에, knowledge graph와 ontology의 구분은 IBM 설명에 근거했다. Palantir Ontology의 제품적 의미는 Palantir 공식 문서에 의존했다. 26

반도체 검사·계측 AI 분석 업무와의 연결점
이 구분은 반도체 현장에서도 유용하다. 예를 들어 “Wafer”, “Defect”, “Inspection”, “Recipe”라는 단어를 팀마다 조금씩 다르게 쓰면 모델 성능이 높아도 분석 결과의 재사용성이 떨어진다. taxonomy 수준의 용어 정리만으로는 충분하지 않다. 어떤 결함이 어떤 검사에서, 어떤 좌표계로, 어떤 Layer/Stack 가정 하에, 누구의 승인으로, 어떤 후속 액션으로 이어지는지까지 명시되어야 실제 운영 체계가 된다. 그것이 온톨로지적 사고가 필요한 이유다. 27

청중이 헷갈릴 수 있는 지점과 방어 논리
“온톨로지는 결국 좋은 스키마 아닌가?”라는 질문이 자주 나온다. 좋은 질문이지만 답은 “부분적으로만 그렇다”이다. 스키마는 저장 구조를 설계한다. 온톨로지는 그 구조가 무엇을 의미하는지, 어떤 관계와 제약 아래 읽혀야 하는지를 정의한다. 또 “knowledge graph면 ontology가 된 것 아닌가?”라는 질문도 생긴다. 하지만 graph는 구현·저장·연결의 관점이고, ontology는 의미 규약의 관점이다. Palantir Ontology는 거기에 액션과 권한까지 더한다. 따라서 같은 그림처럼 보여도 책임 범위가 다르다. 28

시각화 아이디어
세 개의 동심원 또는 층위 도식이 좋다. 가장 안쪽에 존재론, 그다음에 정보학 온톨로지, 가장 바깥에 팔톨로지를 두고, 바깥으로 갈수록 “표현 → 운영”의 범위가 넓어진다는 점을 보여주면 된다. 보조 표로는 taxonomy–schema–ontology–knowledge graph의 차이를 한 장에 비교한 요약표가 효과적이다. 28

의미장으로 읽는 회사의 온톨로지

핵심 메시지
마르쿠스 가브리엘의 의미장 이론은 “같은 객체가 왜 회사 안에서 서로 다른 방식으로 존재하는가”를 설명하는 데 도움이 된다. 이 이론을 그대로 기업 소프트웨어 설계 원리로 옮길 수는 없지만, 적어도 좋은 온톨로지가 “하나의 진짜 정의”를 강요하는 작업이 아니라 “여러 업무 맥락에서 같은 대상을 어떻게 읽을지 정리하는 작업”임을 이해하게 만든다. 29

가브리엘 관련 2차 문헌 요약에 따르면, 그는 “세계 전체”를 하나의 총체적 객체로 보는 관점을 거부하고, 존재를 특정한 의미장 안에서의 출현으로 이해한다. 즉 어떤 것이 존재한다는 말은 그것이 특정한 맥락, 규칙, 구별 기준 속에서 드러난다는 뜻에 가깝다. 이 점에서 그의 ontology는 다원적이며, 하나의 전역 시점으로 모든 것을 정리하려는 충동을 경계한다. 30

이 관점을 회사에 옮기면, 객체는 하나인데 의미장은 여러 개라는 말이 된다. 예를 들어 Wafer는 생산 의미장에서는 공정 이동과 설비 이력의 단위다. 검사·계측 의미장에서는 결함 지도, 계측값, 이미지, 좌표가 귀속되는 관찰 단위다. 수율 의미장에서는 불량 분포와 최종 품질 결과가 집계되는 단위가 된다. 품질·이상탐지 의미장에서는 trend, anomaly

score, root-cause hypothesis, engineer feedback가 연결되는 판단 단위가 된다. **Wafer가 바뀌는 것이 아니라, Wafer를 둘러싼 의미의 좌표계가 바뀌는 것**이다. 이 설명은 철학의 "의미장"을 기업 데이터 설계에 비유적으로 적용한 해석이다. [31]

반도체 공정에서는 이런 다층 의미가 특히 강하다. Hitachi High-Tech의 설명에 따르면 반도체 제조에는 통상 400~600개의 공정 단계가 있고, 전체 소요 기간이 한두 달에 이른다. 초기에 생긴 결함이 뒤 단계에서 더 큰 비용 손실로 이어질 수 있기 때문에, 같은 Wafer를 공정, 검사, 계측, 수율, 품질 대응의 여러 관점으로 동시에 읽어야 한다. 바로 이 때문에 "하나의 테이블 주도형 정의"만으로는 충분하지 않다. [32]

이 프레임은 반도체 검사·계측 AI 조직에 특히 잘 맞는다. 예를 들어 BF/DF 결함 좌표와 SAT void의 공간적 유사성을 보려면, 단순히 이미지 파일끼리 비교하는 것이 아니라, 같은 Wafer인지, 어떤 Layer/Stack 가정을 쓰는지, 검사 시점 차이가 있는지, 어느 공정 step 이후 데이터인지, 이전 유사 case가 있는지를 함께 봐야 한다. 의미장은 곧 데이터 join 조건 이상의 것이다. 그것은 "무엇을 같은 것으로 볼 것인가"에 대한 조직적 합의다. [33]

여기서 중요한 결론은 하나다. 좋은 회사 온톨로지는 DB 테이블을 예쁘게 만드는 일이 아니다. **조직이 실제로 일하는 관점, 즉 의미장을 명시하는 작업**이다. 생산팀, 검사팀, 수율팀, 공정팀, AI팀이 같은 단어를 다른 의미로 쓰는 순간, 데이터는 있어도 판단 체계는 생기지 않는다. 반대로 의미장이 명확해지면, 같은 객체를 서로 다른 맥락에서 보더라도 조직적으로 연결된 판단이 가능해진다. [34]

객관적 근거와 출처
마르쿠스 가브리엘의 "no-world"와 fields of sense의 개요는 원서 소개 및 신뢰 가능한 서평·인터뷰 자료에 기초했다. 반도체 공정의 다단계·장주기 특성은 Hitachi High-Tech의 반도체 제조 설명에 근거했다. 이 둘을 연결하는 것은 Palantir의 공식 철학이 아니라 본 보고서의 해석적 프레임이다. [4]

반도체 검사·계측 AI 분석 업무와의 연결점
이 관점은 특히 cross-functional collaboration 문제를 푸는 데 유용하다. 엔지니어가 "이 wafer가 이상하다"고 말할 때, 그 말이 defect density의 이상인지, SAT void의 군집 이상인지, 특정 equipment-chamber 조합 이후의 편차인지, 혹은 최종 수율과 연결된 문제인지가 분명해야 한다. 의미장을 명시하지 않으면 같은 회의실 안에서 서로 다른 문제를 말하면서도 같은 문제를 토론한다고 착각하게 된다. 온톨로지는 바로 그 오해 비용을 줄이는 도구가 될 수 있다. [34]

청중이 헷갈릴 수 있는 지점과 방어 논리
"이건 철학을 너무 끌어다 쓰는 것 아닌가?"라는 반응이 나올 수 있다. 그 반응은 타당하다. 그래서 이 섹션의 목적은 철학적 권위를 빌려 제품을 정당화하려는 것이 아니다. 오히려 반대로, 조직 온톨로지를 지나치게 기술적 스키마 문제로 축소하지 않기 위한 설명 장치다. 가브리엘과 Palantir 사이에 직접적 연계가 있다는 주장을 하는 것이 아니라, **의미장**이라는 개념이 기업 내부의 다중 관점을 설명하는 데 유용하다는 점을 말하는 것이다. [35]

시각화 아이디어
하나의 Wafer를 가운데 두고, 바깥에 생산·검사·수율·품질·AI 분석의 다섯 의미장을 원형으로 배치하는 그림이 가장 직관적이다. 각 의미장마다 연결되는 객체를 다르게 적으면 좋다. 예를 들어 생산은 Equipment/Recipe/Step, 검사는 Inspection/Defect/Image, 수율은 Bin/Yield/Disposition, 품질은 Case/Feedback/Action, AI는 Score/Embedding/Similar Case를 붙이는 방식이다. [36]

팔톨로지의 구조와 기존 시스템과의 차이

핵심 메시지
Palantir Ontology의 포인트는 객체를 예쁘게 모델링하는 데 있지 않다. 핵심은 **object–link–function–action–security**를 한 체계로 묶어, 사람이든 에이전트든 조직의 "합법적 행동 범위" 안에서 판단과 실행을 하게 만드는 데 있다. 이 점이 ERP, MES, BI, MLOps, 일반 Knowledge Graph와 갈라지는 지점이다. [37]

Palantir 공식 문서를 기준으로 구조를 요약하면 다음과 같다. object type은 현실의 엔티티나 이벤트의 스키마 정의이고, object는 그 인스턴스다. property는 객체의 특성이고, link type은 두 객체 유형 간 관계 정의다. action type은 사용자가 한 번에 일으킬 수 있는 변경 집합이며, function은 운영 문맥에서 빠르게 실행되는 서버 측 로직이다. 또 Ontology는 backing datasource 위에 앉아 동작하며, object/property 수준의 세밀한 권한, edit history, action log, lineage를 포함한 보안·감사 체계를 가진다. AIP는 이 위에서 객체 질의, 함수 실행, 액션 실행, 문서·온톨로지 기반 retrieval을 수행한다. [38]

다음 표는 이를 반도체 업무에 맞춰 풀어쓴 비유다. 표 자체는 Palantir 공식 개념을 반도체 검사·계측 맥락으로 재구성한 요약이다. [39]

팔톨로지 구성요소	쉬운 비유	반도체 예시
Object	명사	Lot, Wafer, Inspection, Defect, Equipment, Recipe
Property	형용사/상태	wafer_id, layer, defect_count, timestamp, score
Link	관계	Wafer belongs_to Lot, Inspection measures Wafer
Function	판단 로직	이상 탐지, 유사 wafer 검색, defect clustering
Action	동사	case open, feedback request, re-inspection request
Security/Governance	문법 규칙	누가 무엇을 보고 실행할 수 있는가, 무엇이 기록되는가

전통적 데이터 모델과의 차이도 분명하다. Palantir 문서는 object type을 dataset, object를 row, property를 column, link type을 join에 비유한다. 여기까지만 보면 일반적 데이터 모델처럼 보인다. 하지만 Ontology overview와 actions 문서는 여기에 kinetics, 즉 변화와 의사결정 프로세스를 함수와 액션으로 얹는다고 설명한다. 다시 말해 **전통적 데이터 모델이 '무엇이 저장되는가'를 강하게 다룬다면, Palantir Ontology는 '무엇이 변경될 수 있으며, 누구에 의해 어떻게 변경되는가'까지 함께 다룬다.** [40]

이 차이는 LLM과 결합될 때 더 크게 드러난다. Palantir의 AIP 관련 공식 문서들은 Ontology가 인간과 에이전트 모두에게 읽을 수 있는 "명사와 동사"의 모델을 제공한다고 설명한다. AIP Chatbot Studio는 LLM에게 object query tool, action tool, document context, ontology context를 설정할 수 있고, AIP Analyst는 함수와 액션을 실행하되 액션은 승인 후 수행되며 되돌릴 수도 있다고 설명한다. 따라서 LLM이 단독으로 기업 업무를 수행하기 어려운 이유는, 언어 생성 능력이 부족해서라기보다 **조직 내부의 객체 정의, 허용된 관계, 가능한 액션, 권한, 감사 경로를 모른 채로는 안전하게 행동할 수 없기 때문**이라고 보는 편이 정확하다. 이 문장은 공식 문서에 대한 해석이다. [41]

기업 시스템과의 비교도 같은 방향으로 읽으면 명확해진다. 아래 표는 일반 기업 시스템의 대표 역할을 요약한 것이다. 이것은 여러 시스템에 대한 일반적 특성과 Palantir 문서의 Ontology 설명을 결합한 실무적 비교표다. [42]

시스템	주 역할	상대적 한계
ERP	자원, 회계, 구매, 재고 관리	업무 기록과 통제에 강하지만 분석·액션 루프는 별도 설계 필요
CRM	고객, 영업, 계약 관리	고객 접점에 강하지만 생산·품질 의미장은 약함
MES	생산 실행, 설비, 공정 이력 관리	제조 실행 기록 중심, 분석·피드백 loop는 별도
BI/Data Mart	지표 조회, 리포팅	분석과 운영이 분리되기 쉬움
MLOps	모델 배포와 관리	모델 수명주기에는 강하지만 업무 객체·권한·조치 문맥은 약함

시스템	주 역할	상대적 한계
Knowledge Graph	지식과 관계 표현	운영 action과 권한 통합은 보통 추가 구현이 필요
Ontology 기반 운영 플랫폼	객체–관계–판단–액션 연결	강력하지만 설계 부담과 조직 합의 비용이 큼

그래서 “ERP/MES는 회사가 한 일을 기록하고, Ontology는 회사가 무엇을 보고 어떻게 판단하고 어떤 행동을 해야 하는지를 연결한다”는 문장은 과장이 아니다. 물론 ERP와 MES도 워크플로와 권한을 가진다. 하지만 대개 그 권한과 워크플로는 애플리케이션 경계 안에 고정되어 있다. Palantir Ontology는 공식 문서상 객체·링크·함수·액션·보안을 하나의 공통층으로 두어, 여러 앱과 AI가 같은 운영 문법을 공유하도록 설계된다는 점이 다르다. [43]

객관적 근거와 출처
이 섹션은 Palantir 공식 Ontology 문서, AIP 문서, object permissioning 문서, functions/actions 문서, edit-history/action-log 문서에 거의 전적으로 의존했다. 기존 기업 시스템과의 비교는 공통적인 시스템 역할에 대한 실무적 요약이며, Palantir 문서가 주장하는 Ontology의 책임 범위와 대비해 정리했다. [44]

반도체 검사·계측 AI 분석 업무와의 연결점
반도체 검사·계측 조직은 보통 이미 MES, SPC, 이미지 저장소, 분석용 notebook, 대시보드, 모델 서빙 환경을 각각 가지고 있다. 문제는 이들이 한 operational loop를 이루지 못하는 경우가 많다는 점이다. 예를 들어 특정 wafer anomaly가 대시보드에 뜨더라도, 그 결과가 자동으로 case object를 만들고, 해당 step owner에게 feedback request를 보내고, 관련 inspection 이미지와 과거 유사 case를 묶고, 인간 승인 후 hold/review action을 실행하는 구조로 닫히지 않으면 조직 지식으로 축적되기 어렵다. 팔톨로지는 바로 이런 “분석과 조치의 연결 계층”이라는 관점에서 의미가 있다. [45]

청중이 헷갈릴 수 있는 지점과 방어 논리
“MES도 액션이 있고 ERP도 승인 프로세스가 있는데 왜 Ontology가 다른가?”라는 질문이 나올 수 있다. 방어 논리는 간단하다. 기존 시스템에도 액션은 있다. 그러나 그 액션은 보통 해당 시스템이 정의한 엔터티와 화면, 승인 흐름 안에 갇혀 있다. 반면 Ontology 기반 접근은 공통 객체 계층에서 액션과 함수를 재사용하게 한다. 또 “Knowledge Graph만 잘 만들면 되는 것 아닌가?”라는 반론도 가능하다. 하지만 많은 graph 시스템은 운영 액션, 세밀 권한, 승인 후 실행, edit history, immutable audit trail을 별도 제품이나 추가 구현에 의존한다. Palantir는 이를 제품 레벨에서 한데 묶는다. [46]

시각화 아이디어
가장 효과적인 그림은 두 장이다. 첫째, Object–Link–Function–Action–Security를 좌에서 우로 배치한 구조도. 둘째, “LLM alone”과 “LLM + Ontology”를 대비한 비교도다. 전자는 자연어 추론은 가능하지만 권한·객체·허용된 액션·감사 경로가 약하다고 두고, 후자는 object query, action approval, provenance, audit trail이 붙는 구조를 보여주면 된다. [41]

반도체 검사·계측 AI 조직에서의 적용과 개발 절차

핵심 메시지
반도체 검사·계측 AI 조직이 온톨로지를 해야 하는 이유는 “팔란티어처럼 보이기 위해서”가 아니다. 이유는 훨씬 실무적이다. 검사·계측 AI의 가치는 모델 성능의 절대치보다, **분석 결과가 조직의 판단과 행동으로 이어지고 다시 학습과 운영 개선으로 돌아오는지**에 달려 있기 때문이다. [47]

반도체 제조는 다종 데이터가 얽힌 환경이다. Hitachi High-Tech는 검사와 계측이 긴 공정 구간 전체의 관리에 중요하다고 설명한다. 최근 문헌들은 wafer map anomaly detection, spatial defect clustering, SEM 기반 defect inspection, scratch defect detection 등 다양한 AI 과제가 이미 활발하다고 보여준다. 그러나 동시에 제조 데이터는 이질적이고 siloed되어 있으며, 의미와 지식이 형식화되지 않으면 시스템 수준의 ad hoc analysis와 의사결정이 어렵

다고 제조 knowledge graph 연구는 지적한다. 반도체 도메인 자체도 semantics-aware data management가 부족하다는 지적이 있다. [48]

이 맥락에서 Ontology Development 101의 첫 원칙은 **use case first**다. Palantir도 AIP Bootcamp를 "0에서 use case까지 5일"이라는 방식으로 설명한다. 반도체 검사·계측 조직이 따라야 할 것도 같은 원리다. "회사 전체 데이터를 온톨로지화하자"는 접근은 실패 확률이 높다. 대신 "이상 case open → 분석 → 피드백 요청 → 조치 → case close"처럼 닫힌 operational loop 하나를 먼저 설계해야 한다. [49]

실무적으로는 다음 정도의 최소 모델이 적당하다.
Object 후보는 Lot, Wafer, Product, Process Step, Equipment, Chamber, Recipe, Inspection, Measurement, Defect, Defect Cluster, Image, SAT Void, Layer, Stack, Analysis Case, Feedback Request, Root-cause Hypothesis, Action Item, User/Department/Owner다.
Link 후보는 Wafer belongs_to Lot, Wafer processed_by Equipment, Inspection measures Wafer, Defect detected_in Inspection, Image represents Defect, SAT Void overlaps_with Defect Cluster, Analysis Case investigates Defect Trend, Feedback Request assigned_to Process Owner, Action Item resolves Analysis Case다.
Function 후보는 wafer anomaly detection, defect clustering, BF/SEM image classification, SAT void overlap analysis, similar case retrieval, root-cause candidate ranking, adaptive threshold recommendation, feedback summary generation이다.
Action 후보는 abnormal case open, similar case recommend, process owner feedback request, re-inspection request, hold/release review, result approval, case close, training-data reflect다.
이 구성은 반도체 검사·계측의 일반적 업무 흐름과 Palantir 공식 object/action/function 개념을 결합한 제안이다. [50]

거버넌스는 처음부터 같이 들어가야 한다. Palantir 문서 기준으로 object/property 수준 보안, project-based permissions, provenance, edit history, immutable audit trail, lineage가 모두 별도 기능으로 존재한다. 반도체 현장에서는 이를 권한, 데이터 lineage, action log, audit trail, 모델 버전, human approval, 보안 등급, 부서별 visibility로 번역하면 된다. 즉 governance는 나중에 붙이는 옵션이 아니라 ontology의 기본 설계 항목이다. [51]

구체적 use case를 반도체 검사·계측 맥락에서 보면 다음과 같다.
첫째, **BF/DF 검사 결과와 SAT void 결과의 위치적 유사성 분석**이다. 문제는 서로 다른 검사 결과를 단일 이미지 과제로 볼 수 없다는 데 있다. 같은 Wafer, Layer, Stack, 좌표 변환, 시간차, defect cluster context를 함께 묶어야 한다. Ontology는 이질적 검사 결과를 하나의 case 중심 의미장으로 묶어, "어떤 void를 우선 검토할 것인가"를 판단 단위로 바꿔준다. 수학적 유사도 그 자체보다 중요한 것은 **유사도의 귀속 대상이 무엇인지**를 명확히 하는 일이다. 이 점은 SEM defect inspection 및 제조용 knowledge graph 기반 품질 판단 연구와 잘 맞닿아 있다. [52]

둘째, **Wafer defect map anomaly detection**이다. wafer map 이상 탐지와 unseen pattern 탐지는 이미 활발한 연구 주제다. 하지만 모델이 anomaly score를 내는 것만으로는 현장 업무가 끝나지 않는다. 점수가 임계값을 넘으면 Analysis Case를 열고, 해당 Step/Equipment/Recipe owner에게 feedback request를 보내고, 결과에 따라 재검사 또는 hold review로 이어져야 한다. 이후 실제 조치 결과가 threshold와 모델 개선으로 돌아가야 가치가 커진다. 즉 anomaly detection은 모델 문제가 아니라 operational loop 문제다. [53]

셋째, **BF/SEM image classification과 review coordinate 자동화**다. 최근 연구들은 SEM 기반 defect inspection과 표면 scratch detection 등에서 AI의 가능성을 보여준다. 그러나 분류 모델이 "defect class A"를 내놓는 순간 바로 종료되면, 엔지니어의 review point 확정, 실제 equipment 상태, process step 맥락, 잘못된 추천에 대한 피드백, 데이터셋 반영까지 이어지지 않는다. Ontology는 image–defect–review point–equipment–process step의 연결을 명시하고, AI 추천에 대한 인간 피드백을 구조화된 edit로 저장하게 해준다. [54]

넷째, **유사 과거 case retrieval**이다. 제조 품질관리 knowledge graph 연구는 case retrieval, 자동 분석, 의사결정 지원이 ontology+graph 기반으로 가능함을 보여준다. 반도체 검사·계측에서도 현재 wafer/defect trend와 비슷한 과

거 case를 찾아 root-cause hypothesis를 생성하는 구조가 유용하다. 여기서 중요한 것은 단순 벡터 유사 검색이 아니라, “같은 product family인가, 같은 layer family인가, 같은 equipment class인가, 같은 불량 현상인가” 같은 구조적 유사성을 함께 보는 것이다. 이것이 조직의 암묵지를 명시지로 축적하는 핵심 경로다. 55

현실적인 PoC 범위도 너무 크게 잡을 필요가 없다. 4~8주 안에 가능한 최소 범위는 “검사·계측 이상 case open → 분석 → feedback request → 조치 → case close” 흐름 하나다. 이때 성공 기준은 모델 F1 하나가 아니라, **분석 결과가 실제 판단과 action으로 이어졌는가**, **누가 무엇을 보고 어떤 조치를 했는가가 남는가**, **동일 유형 case가 이전보다 빨리 재현되고 처리되는가**다. 이 기준이야말로 operational AI의 성숙도를 보는 더 좋은 지표다. 56

객관적 근거와 출처
이 섹션은 반도체 검사·계측 기술 문헌, 제조 knowledge graph/ontology 연구, Palantir의 use-case-first 제품 채택 방식, 그리고 Palantir의 action/governance 문서를 결합해 작성했다. 특히 반도체 도메인의 의미 관리 부족은 SemicONTO, 제조 ad hoc analysis의 어려움은 Journal of Intelligent Manufacturing 논문, case retrieval와 assisted decision making은 Engineering 저널 논문을 근거로 삼았다. 57

반도체 검사·계측 AI 분석 업무와의 연결점
이 보고서의 직접적인 메시지는 여기 있다. 검사 데이터는 이미지형, 공간형, 시계열형, 관계형 데이터가 한꺼번에 섞인 복합 의미장이다. 따라서 AI 분석 결과는 Wafer, Lot, Step, Equipment, Recipe, Defect, Inspection, Feedback, Action과 연결될 때 가치가 커진다. dashboard 중심 분석이 “보기”를 돕는다면, ontology 기반 operational AI는 “판단과 실행”을 돕는다. 반도체 AI 팀이 온톨로지를 해야 하는 이유는 모델을 포장하기 위해서가 아니라, **모델 결과를 조직의 재사용 가능한 판단 체계로 바꾸기 위해서**다. 58

청중이 헷갈릴 수 있는 지점과 방어 논리
“결국 이것도 또 하나의 지식관리 프로젝트 아니냐”는 의심이 생길 수 있다. 그 의심이 틀린 것은 아니다. 온톨로지 프로젝트는 실제로 지식관리의 성격을 가진다. 하지만 차이는 산출물이 문서나 위키에 머무르지 않는다는 점이다. object, link, function, action, permission으로 구현되면 그 지식은 검색 가능한 문장보다 **실행 가능한 운영 구조**가 된다. 또 “먼저 모델을 잘 만들고 나중에 구조화해도 되지 않나?”라는 질문도 가능하다. 그러나 현장에서는 오히려 반대인 경우가 많다. 구조 없는 모델은 PoC에서는 빛나도 운영에서는 증발하기 쉽다. 59

시각화 아이디어
보고서 보강용으로는 두 그림이 효과적이다. 하나는 “이상 case open → 분석 함수 → feedback request → 승인/조치 → case close → 재학습”의 closed loop 흐름도다. 다른 하나는 BF/DF–SEM–SAT–Metrology 데이터를 Wafer/Layer/Defect 중심으로 연결한 멀티모달 의미장 다이어그램이다. use case별로는 Before/After 비교가 좋다. “대시보드 중심 분석”과 “ontology 기반 operational AI”를 나란히 놓고, 후자에만 case object, action log, feedback loop가 붙는 구조를 보여주면 된다. 60

비판적 관점

핵심 메시지
온톨로지는 강력하지만 만능이 아니다. 특히 Palantir Ontology 같은 기업형 ontology 접근은 구축 비용, 조직 저항, 도메인 전문가 의존성, legacy 통합, 데이터 품질, 권한·감사 체계의 복잡성 때문에 쉽게 실패할 수 있다. 더 중요한 점은, ontology가 있다고 AI가 자동으로 좋아지는 것은 아니라는 사실이다. Ontology는 모델이 아니라 **운영 체계**에 가깝다. 61

첫째 문제는 **구축 비용과 조직 저항**이다. 제조 knowledge graph 연구는 ad hoc analysis가 어려운 이유로 데이터 이질성, 정보 사일로, 지식 형식화 부족을 꼽는다. Ontology는 바로 이 문제를 풀려고 등장하지만, 역설적으로 그 문제를 해결하려면 여러 팀이 자신의 용어와 업무 정의를 다시 합의해야 한다. 이는 기술 프로젝트이면서 동시에 조직 설계 프로젝트다. 따라서 “좋은 모델러 한 명”으로 해결되지 않는다. 62

둘째는 **도메인 전문가 의존성**이다. 반도체 도메인에서 semantics-aware data management가 부족하다는 SemicONTO의 지적은 곧, 표준 형태의 재사용 가능한 개념 체계가 아직 충분히 성숙하지 않았다는 뜻이기도 하다. 이런 환경에서는 Ontology 설계가 AI팀만의 일이 될 수 없다. 공정, 검사, 계측, 품질, 수율, IT, 보안 담당자가 실제 객체 정의와 예외 규칙을 넣어주지 않으면 온톨로지가 현실을 잘못 반영할 가능성이 높다. 63

셋째는 **잘못 정의된 ontology의 경직성**이다. 온톨로지는 표준화의 도구지만, 잘못 설계하면 현장의 유연성을 죽일 수도 있다. 예를 들어 layer 해석이 아직 불확실한데 지나치게 확정적인 link만 허용하면, 현장의 불확실성이 시스템 밖의 메모와 구두 보고로 다시 빠져나간다. 좋은 ontology는 모든 것을 강제하는 체계가 아니라, 불확실성과 가설도 객체화하거나 상태로 표현할 수 있어야 한다. 이것은 본 보고서의 실무적 판단이지만, 제조의 ad hoc analysis 요구사항과 knowledge graph 기반 의사결정 문헌이 보여주는 교훈과도 일치한다. 34

넷째는 **legacy 통합과 권한 체계의 복잡성**이다. Palantir의 보안 문서만 봐도 object/resource permissions, object/property security policies, datasource policies, project-based permissions, markings, role grants, edit history, action log, lineage가 각각 분리된 관리 지점으로 존재한다. 이는 강력한 장점이면서 동시에 도입 난이도가 높다는 증거다. 반도체 조직도 마찬가지다. MES 권한, 이미지 저장소 권한, 연구용 분석 공간, 고객/제품 보안 등급, 부서별 visibility가 얽히면 ontology는 기술보다 governance 설계가 더 어려워질 수 있다. 64

다섯째는 **데이터 품질 문제**다. ontology는 나쁜 데이터를 좋은 데이터로 바꾸지 못한다. object–property–link를 아무리 잘 정의해도, 좌표계 불일치, layer mapping 오류, timestamp drift, equipment metadata 누락, 라벨 품질 저하는 그대로 남는다. 반도체 defect inspection과 wafer map anomaly detection 연구가 보여주듯, 실제 모델 성능과 운영 성과는 여전히 데이터 품질과 레이블링 전략에 크게 좌우된다. Ontology는 이 문제를 보이게 하고 연결할 수는 있지만 자동 해결하지는 않는다. 65

여섯째는 **ontology가 있다고 AI가 자동으로 좋아지지 않는다는 점**이다. Palantir AIP 역시 Evals, governance, approval, action execution을 별도로 강조한다. 이는 LLM이 온톨로지를 얻었다고 해서 곧바로 신뢰 가능한 의사결정 주체가 되는 것이 아니라, 평가와 승인, 감사 가능한 실행 경로가 함께 마련되어야 한다는 뜻이다. AI ethics/gov 문서도 개발 과정, 데이터 소스, provenance의 문서화를 강조한다. 즉 ontology는 AI 성능 부스터라기보다, AI가 기업 안에서 다닐 수 있는 도로망과 교통법규에 가깝다. 66

객관적 근거와 출처
이 섹션의 근거는 제조 knowledge graph의 한계 진단, 반도체 semantics-aware data management의 부족, Palantir의 보안·감사·edit-history·lineage·AI governance 문서다. 중요한 점은, Palantir의 복잡한 권한·감사 문서 자체가 이 접근이 결코 "쉽게 되는 마법"이 아님을 보여준다는 사실이다. 67

반도체 검사·계측 AI 분석 업무와의 연결점
반도체 현장에서는 특히 "불완전한 지식"을 다루는 능력이 중요하다. 예를 들어 실제 root cause는 나중에 뒤집힐 수 있고, inspection–metrology 간 좌표 매칭도 조건부일 수 있다. 이런 현실을 무시하고 ontology를 너무 일찍 닫아버리면 시스템은 깔끔해지지만 현장은 시스템 밖으로 도망간다. 그래서 반도체용 ontology는 확정적 진실의 사전이 아니라, **객체·증거·가설·조치·피드백을 함께 기록하는 운영 장부**에 가까워야 한다. 이 점을 놓치면 "현장성이 없는 깔끔한 모델"이 만들어질 가능성이 크다. 68

청중이 헷갈릴 수 있는 지점과 방어 논리
비판을 제기하면 "그럼 하지 말자는 얘기냐"는 반응이 나올 수 있다. 그렇지는 않다. 오히려 반대다. ontology는 강력하기 때문에 더 작게, 더 명시적으로, 더 governance-aware하게 시작해야 한다는 뜻이다. 이 비판 섹션의 목적은 Palantir를 깎아내리려는 것도, ontology를 포기하자는 것도 아니다. **온톨로지는 성능 향상 요술봉이 아니라 운영 체계 설계 도구**라는 점을 분명히 하려는 것이다. 69

시각화 아이디어
"온톨로지의 장점"만을 나열한 그림보다, 양면 리스트가 더 좋다. 왼쪽에는 semantic consistency, actionability, traceability, AI readiness를 두고, 오른쪽에는 cost, expert dependency, legacy integration, security

complexity, data quality dependence를 놓는 방식이다. 또는 2×2 매트릭스에서 “가치가 큰데 도입 난이도도 큰 항목”으로 governance와 cross-functional semantics를 표시하는 방식도 좋다. [70]

결론과 출처

그래서 우리는 내일부터 무엇을 작게 시작할 수 있는가
가장 좋은 시작점은 전사 ontology가 아니다. **하나의 닫힌 use case**다. 반도체 검사·계측 조직이라면 “검사·계측 이상 case open → 분석 → feedback request → 조치 → case close” 흐름 하나만 잡아도 충분하다. 이 흐름에 필요한 최소 Object는 Wafer, Inspection, Defect, Analysis Case, Feedback Request, Action Item 정도이고, 최소 Link는 “Inspection measures Wafer”, “Defect detected_in Inspection”, “Case investigates Wafer”, “Feedback Request assigned_to Owner” 정도면 된다. 최소 Function은 anomaly scoring, similar case lookup, feedback summary generation 정도이고, 최소 Action은 case open, feedback request, approve/close 정도면 된다. 이것이 4~8주 PoC의 현실적 범위다. [71]

둘째, 기존 dashboard를 버릴 필요도 없다. 오히려 더 현실적인 방법은 **기존 dashboard에 action log와 feedback loop를 붙이는 것**이다. 알람을 보는 화면 옆에 “case 생성”, “담당 요청”, “검토 상태”, “최종 조치”, “재학습 반영 여부”를 객체와 액션으로 남기기 시작하면, 대시보드는 단순 조회 도구에서 operational console로 진화한다. Palantir 문서가 edit history와 immutable audit trail을 강조하는 이유도 여기에 있다. 분석 결과가 행동으로 이어지고, 그 행동이 다시 데이터로 남아야 조직 지식이 된다. [72]

셋째, 성공 기준도 바꿔야 한다. PoC의 성공을 “모델 정확도 2% 개선”으로만 두면, 결국 또 하나의 모델 데모가 될 가능성이 높다. 더 나은 성공 기준은 다음에 가깝다. **이상 이벤트 중 얼마가 실제 case로 전환되었는가. case 처리 시간이 줄었는가. 유사 과거 사례를 재사용했는가. feedback가 구조화되어 다시 모델 개선에 들어갔는가. 승인·조치 이력이 감사 가능하게 남았는가.** 이런 지표가야말로 operational AI가 조직에 스며드는지를 보여준다. [60]

결국 우리가 내일부터 시작할 일은 거창하지 않다. “우리 팀이 자주 다루는 이상 case 하나를 골라, 그 case를 구성하는 객체·관계·판단 로직·액션·권한을 한 장에 적어보는 것”이다. 그 한 장이 쌓이면, 모델은 더 이상 일회성 분석 스크립트가 아니라 조직의 판단 체계 일부가 된다. 이 보고서가 말하고 싶은 결론은 바로 그것이다. **온톨로지는 모델을 대신하지 않는다. 그러나 모델이 조직 안에서 살아남게 만드는 운영 골격이 될 수 있다.** [73]

출처 목록

공식 출처

- Palantir Technologies, 2025 FY PLTR 10-K; 제품군, 창업 배경, 세그먼트 매출, 지역 매출, 사업 개요. [9]
- Palantir Technologies, Q1 2026 Business Update; 2026년 1분기 매출 성장, 고객 수, 미국 상업 성장, 대형 딜 현황. [11]
- Palantir Foundry Documentation, Integrated platforms: AIP, Foundry, and Apollo; Enterprise Operating System 구조. [74]
- Palantir Foundry Documentation, Ontology overview 및 Core concepts; Ontology를 operational layer, digital twin, object/property/link/action/function 구조로 설명. [75]
- Palantir Foundry Documentation, Functions overview, Action types overview, Object permissioning, Managing object security; 함수, 액션, object/property 보안, cell-level security. [76]
- Palantir Foundry Documentation, AIP overview, AIP architecture overview, AIP Chatbot Studio tools/retrieval context, AIP Analyst overview; AIP와 Ontology의 결합, 함수·액션 실행, object query, approval. [77]
- Palantir Foundry Documentation, Enable user edit history, Edit History widget, Action log, Data Lineage, AI ethics and governance; audit trail, provenance, lineage, traceability. [78]
- Palantir, AIP Bootcamp; use-case-first 도입 방식. [79]
- U.S. Department of Defense, Contracts for May 21, 2025; Maven Smart System 수정 계약. [80]

- U.S. Department of Defense, Contracts for Aug. 1, 2025; 미 육군의 Palantir 기업형 계약 공지. [81]
- S&P Dow Jones Indices, Palantir Technologies Set to Join S&P 500 및 S&P 100 Addition 공지. [82]
- 시장 데이터, PLTR finance quote; 2026-06-24 기준 주가와 시가총액.

학술·표준·보조 출처

- Stanford Encyclopedia of Philosophy, Ontology and Information Systems; 철학적 ontology와 정보 시스템 ontology의 구분. [83]
- Stanford Encyclopedia of Philosophy, Ontological Commitment; “무엇이 존재하는가”라는 존재론의 기본 질문. [84]
- Gruber, T., A translation approach to portable ontology specifications 및 정의 해설 자료; ontology의 고전적 정의 계보. [85]
- Guarino, N., What is an Ontology?; taxonomy와 ontology의 관계, 공유·형식성 논의. [86]
- W3C, OWL 2 Web Ontology Language Primer; ontology 언어의 형식적 표현. [87]
- W3C, SKOS Primer; taxonomy와 controlled vocabulary의 표현. [88]
- IBM, What is a Knowledge Graph?; knowledge graph와 ontology의 관계 설명. [24]
- Markus Gabriel, Fields of Sense 원서 소개 및 관련 서평·인터뷰 자료; fields of sense와 no-world 개요. [89]
- Meyers et al., Towards a knowledge graph framework for ad hoc analysis in manufacturing; 제조 데이터 이질성, 사일로, 지식 형식화 문제. [62]
- Li et al., Initial development of an ontology for the semiconductor domain – SemicONTO; 반도체 도메인의 semantics-aware data management 부족. [63]
- Wang et al., An Intelligent Quality Control Method for Manufacturing Processes Based on a Human-Cyber-Physical Knowledge Graph; case retrieval, 자동 분석, assisted decision making. [90]
- Hitachi High-Tech, Semiconductor Metrology and Inspection; 반도체 제조의 다단계 공정과 검사·계측의 중요성. [32]
- Nakazawa et al., Anomaly Detection and Segmentation for Wafer Defect Patterns; wafer map 이상 탐지. [91]
- Yuan et al., Detection of Spatial Defect Patterns Generated in Semiconductor Fabrication Processes; spatial defect clustering. [92]
- Dehaerne et al., SEM-based automatic defect inspection for semiconductor manufacturing; SEM defect inspection 자동화. [93]
- Sánchez Peñaloza et al., Vision-based wafer inspection in semiconductor manufacturing; scratch defect 탐지 사례. [94]

navlist최근 동향 참고 기사turn31news30,turn30news9,turn28news28,turn27news19

---

[1] [6] [8] [9] [10] [15] [16] [19] https://investors.palantir.com/files/2025%20FY%20PLTR%2010-K.pdf
https://investors.palantir.com/files/2025%20FY%20PLTR%2010-K.pdf

[2] [20] [21] [26] [83] https://plato.stanford.edu/entries/ontology-is/
https://plato.stanford.edu/entries/ontology-is/

[3] [7] [17] [43] [69] [73] [75] https://palantir.com/docs/foundry/ontology/overview/
https://palantir.com/docs/foundry/ontology/overview/

[4] [29] [30] [35] https://c-scp.org/2015/08/25/markus-gabriel-fields-of-sense
https://c-scp.org/2015/08/25/markus-gabriel-fields-of-sense

[5] [47] [55] [59] [60] [68] [90] https://www.engineering.org.cn/engi/EN/10.1016/j.eng.2024.03.022
https://www.engineering.org.cn/engi/EN/10.1016/j.eng.2024.03.022

11 https://investors.palantir.com/files/Palantir%20-%20Q1%202026%20Business%20Update.pdf
https://investors.palantir.com/files/Palantir%20-%20Q1%202026%20Business%20Update.pdf

12 80 https://www.defense.gov/News/Contracts/Contract/Article/4194643/
https://www.defense.gov/News/Contracts/Contract/Article/4194643/

13 82 https://press.spglobal.com/2024-09-06-Palantir-Technologies%2C-Dell-Technologies%2C-and-Erie-Indemnity-Set-to-Join-S-P-500-Others-to-Join-S-P-MidCap-400-and-S-P-SmallCap-600
https://press.spglobal.com/2024-09-06-Palantir-Technologies%2C-Dell-Technologies%2C-and-Erie-Indemnity-Set-to-Join-S-P-500-Others-to-Join-S-P-MidCap-400-and-S-P-SmallCap-600

14 18 74 https://palantir.com/docs/foundry/architecture-center/platforms/
https://palantir.com/docs/foundry/architecture-center/platforms/

22 85 https://tomgruber.org/writing/definition-of-ontology.pdf
https://tomgruber.org/writing/definition-of-ontology.pdf

23 88 https://www.w3.org/TR/2008/WD-skos-primer-20080221/
https://www.w3.org/TR/2008/WD-skos-primer-20080221/

24 28 46 https://www.ibm.com/think/topics/knowledge-graph
https://www.ibm.com/think/topics/knowledge-graph

25 37 38 39 40 44 50 71 https://palantir.com/docs/foundry/ontology/core-concepts/
https://palantir.com/docs/foundry/ontology/core-concepts/

27 34 42 58 61 62 67 70 https://link.springer.com/article/10.1007/s10845-023-02319-6
https://link.springer.com/article/10.1007/s10845-023-02319-6

31 32 36 48 https://www.hitachi-hightech.com/global/en/knowledge/semiconductor/room/manufacturing/metrology-inspection.html
https://www.hitachi-hightech.com/global/en/knowledge/semiconductor/room/manufacturing/metrology-inspection.html

33 52 54 93 https://www.spiedigitallibrary.org/journals/journal-of-micro-nanopatterning-materials-and-metrology/volume-24/issue-02/020901/Scanning-electron-microscopy-based-automatic-defect-inspection-for-semiconductor-manufacturing/10.1117/1.JMM.24.2.020901.pdf
https://www.spiedigitallibrary.org/journals/journal-of-micro-nanopatterning-materials-and-metrology/volume-24/issue-02/020901/Scanning-electron-microscopy-based-automatic-defect-inspection-for-semiconductor-manufacturing/10.1117/1.JMM.24.2.020901.pdf

41 https://palantir.com/docs/foundry/architecture-center/aip-architecture/
https://palantir.com/docs/foundry/architecture-center/aip-architecture/

45 https://palantir.com/docs/foundry/object-edits/overview/
https://palantir.com/docs/foundry/object-edits/overview/

49 79 https://www.palantir.com/platforms/aip/bootcamp/
https://www.palantir.com/platforms/aip/bootcamp/

51 https://palantir.com/docs/foundry/object-permissioning/managing-object-security/
https://palantir.com/docs/foundry/object-permissioning/managing-object-security/

53 65 91 https://ieeexplore.ieee.org/document/8634922/
https://ieeexplore.ieee.org/document/8634922/

56 72 https://palantir.com/docs/foundry/workshop/widgets-edits-history/
https://palantir.com/docs/foundry/workshop/widgets-edits-history/

57 63 https://ceur-ws.org/Vol-3760/paper12.pdf
https://ceur-ws.org/Vol-3760/paper12.pdf

64 https://palantir.com/docs/foundry/object-permissioning/overview/
https://palantir.com/docs/foundry/object-permissioning/overview/

66 77 https://palantir.com/docs/foundry/aip/overview/
https://palantir.com/docs/foundry/aip/overview/

76 https://palantir.com/docs/foundry/functions/overview/
https://palantir.com/docs/foundry/functions/overview/

78 https://palantir.com/docs/foundry/object-edits/user-edit-history/
https://palantir.com/docs/foundry/object-edits/user-edit-history/

81 https://www.defense.gov/News/Contracts/Contract/Article/4262594/
https://www.defense.gov/News/Contracts/Contract/Article/4262594/

84 https://plato.stanford.edu/entries/ontological-commitment/
https://plato.stanford.edu/entries/ontological-commitment/

86 https://iaoa.org/isc2012/docs/Guarino2009_What_is_an_Ontology.pdf
https://iaoa.org/isc2012/docs/Guarino2009_What_is_an_Ontology.pdf

87 https://www.w3.org/TR/owl2-primer/
https://www.w3.org/TR/owl2-primer/

89 https://philpapers.org/rec/GABFOS
https://philpapers.org/rec/GABFOS

92 https://ieeexplore.ieee.org/document/5776703/
https://ieeexplore.ieee.org/document/5776703/

94 https://www.researchgate.net/publication/401280160_Vision-based_wafer_inspection_in_semiconductor_manufacturing_a_case_study_on_scratch_defect_detection_using_synthetic_
https://www.researchgate.net/publication/401280160_Vision-based_wafer_inspection_in_semiconductor_manufacturing_a_case_study_on_scratch_defect_detection_using_synthetic_data_and_YOLO_models